Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

İleri Çok Kanallı Programlama

Bölüm 10 – Hafta 10

Yazılım Geliştirme II (BILB2000)

Bahar Dönemi – 10. Hafta

Bu Haftanın Planı

1. Ünite 1: condition_variable ile Koordinasyon
2. Ünite 2: std::async ve Future/Promise
3. Ünite 3: Thread Havuzu Tasarımı
4. Ünite 4: Qt ile Çok Kanallı Programlama

Hafta 10'un Büyük Resmi

Ham primitiflerden üst seviye araçlara

Bölüm 9'da `std::thread`, `mutex`, `atomic` ile eşzamanlılığın temellerini öğrendik. Bu hafta üst seviye araçlarla tanışıyoruz: `condition_variable`, `async/future`, **thread havuzu** ve Qt'nin çok kanallı modülleri.

Kazanımlar

- `condition_variable` ile izlek koordinasyonu
- `async` + `future` ile üst seviye async
- Sıfırdan bir thread havuzu inşası
- Qt Worker Pattern ve `QtConcurrent`

OBYS Bu Hafta

- Kayıt kuyruğu (çoklu işçi)
- JSON export asenkron
- 10.000 öğrenci GPA paralel
- UI donmayan responsive rapor

Ünite 1: condition_variable ile Koordinasyon

Ünite 1 – Öğrenme Hedefleri

Bu ünitede öğreneceğin

- `std::condition_variable` ile wait/notify
- Sahte uyanma (spurious wakeup) ve predicate koruması
- Üretici-tüketici (producer-consumer) deseni
- Sınırlı tampon (bounded buffer)
- OBYS'de çoklu işçili kayıt kuyruğu

Neden condition_variable?

Busy-waiting'in maliyeti

```
std::atomic<bool> veriHazir{false};
void diskIsi() {
    while (!veriHazir.load()) {
        // Bir sey yapma, tekrar kontrol
    }
    diskYaz();
}
```

Sorun

İzlek bir CPU çekirdeğini tam kullanım ile işgal eder. Pil ömrü, fan gürültüsü, CPU saykılları boşa harcanır.

Çözüm

`std::condition_variable` ile izleği *uyut*, olay geldiğinde *uyandır*.

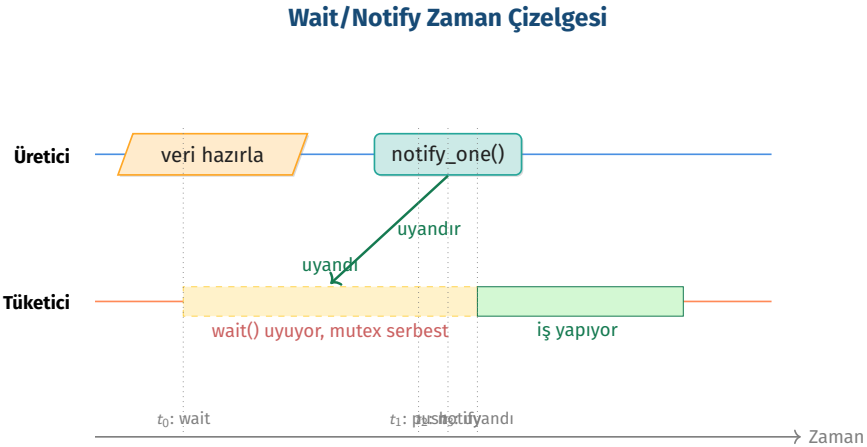
wait / notify Temel Mekaniği

```
std::mutex mtx;
std::condition_variable cv;
bool veriHazir = false;

void tuketici() {
    std::unique_lock<std::mutex> kilit(mtx);
    cv.wait(kilit, [] { return veriHazir; });
    // islem ...
}

void uretici() {
    { std::lock_guard<std::mutex> kilit(mtx);
      veriHazir = true; }
    cv.notify_one();
}
```

Atomik üç işlem

`wait(kilit, predicate)`: (1) mutex'i açar, (2) uyur, (3) uyanınca mutex'i tekrar alır.

wait Zaman Çizelgesi



Dikkat

Sahte Uyanmalar (Spurious Wakeups)

Tanım

İşletim sistemi, hiç notify olmadan `wait`'ten dönebilir. POSIX ve Windows'ta **belgelenmiş** bir olgudur.

Koruma: Predicate Şart!

- `cv.wait(kilit);` – yanlış
- `cv.wait(kilit, []{return hazir;});` – doğru

Standart, predicate'li sürümün şu döngüye eşdeğer olduğunu söyler:

```
while (!predicate()) cv.wait(kilit);
```

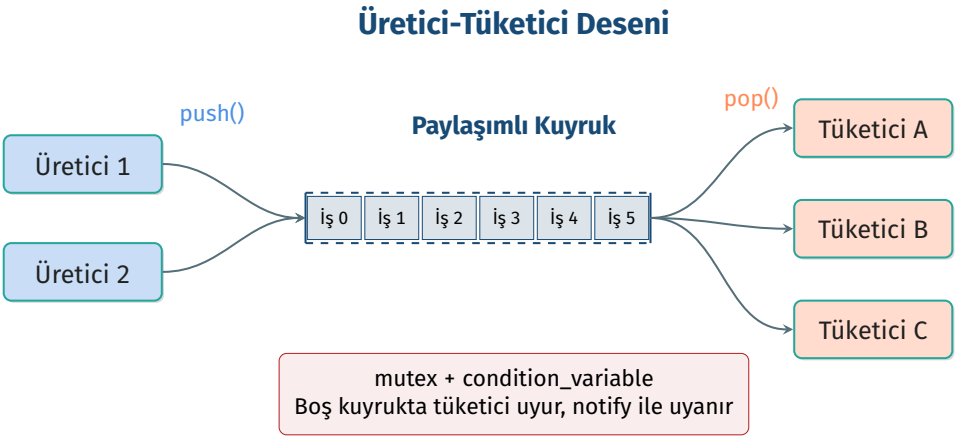
notify_one vs notify_all

notify_one

- Bekleyenlerden *bir* izleği uyandırır
- Hangisi? Zamanlayıcı karar verir
- Tek iş eklendiyse ideal
- Verimli: boşa uyandırma yok

notify_all

- *Tüm* bekleyenleri uyandırır
- Kapatma sinyali için zorunlu
- Toplu iş ekleme için kullanılır
- "Gök gürültüsü sürüsü" etkisi



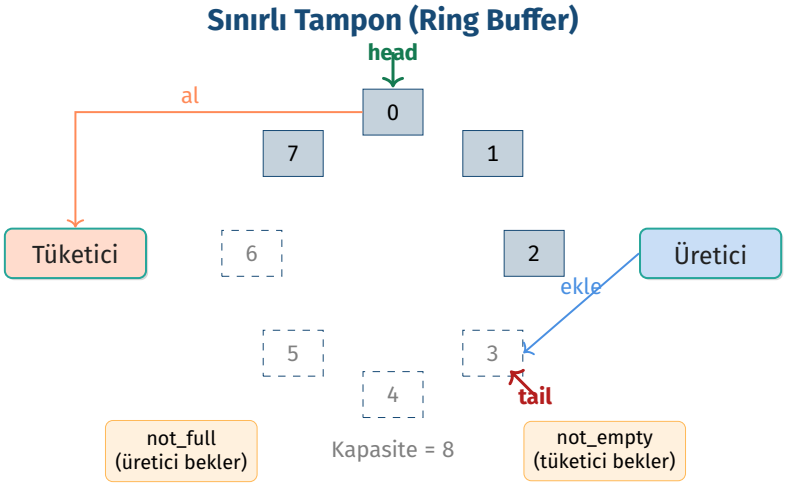
Üç bileşen

`std::queue` + `std::mutex` + `std::condition_variable`

GorevKuyrugu Sınıfı (Özet)

```
class GorevKuyrugu {
    std::queue<Gorev> kuyruk_;
    std::mutex mtx_;
    std::condition_variable cv_;
    std::atomic<bool> kapaniyor_{false};
public:
    void ekle(Gorev g) {
        { std::lock_guard lk(mtx_); kuyruk_.push(std::move(g)); }
        cv_.notify_one();
    }
    bool al(Gorev& g) {
        std::unique_lock lk(mtx_);
        cv_.wait(lk, [this] {
            return !kuyruk_.empty() || kapaniyor_.load(); });
        if (kuyruk_.empty()) return false;
        g = std::move(kuyruk_.front()); kuyruk_.pop();
        return true;
    }
    void kapat() { kapaniyor_ = true; cv_.notify_all(); }
};
```

Sınırlı Tampon (Bounded Buffer)



OBYS: Kayıt Kuyruğu

Senaryo

Kullanıcı 1000 öğrenciyi CSV'den yüklüyor. Disk işlemi yavaş; UI 30 saniye donmamalı. Çözüm: 3 işçi izleği bir kuyruktan iş alıp paralel diske yazar.

Avantajlar

- UI hemen döner (milisaniyeler)
- İşçiler paralel diske yazar (disk hızı sınırı)
- `std::atomic` sayaç ile ilerleme UI'de
- Destructor graceful shutdown

Ünite 1 Özet

Önemli kurallar

1. `wait`'te her zaman predicate kullan
2. `unique_lock` şart (lock_guard değil)
3. notify'ı kilit dışında çağır (verimlilik için)
4. Kapatma bayrağını predicate'e dahil et
5. Çoklu işçi kapatmada `notify_all` kullan
6. Destructor'da `join` unutma

Ünite 2: std::async ve Future/Promise

Ünite 2 – Öğrenme Hedefleri

- `std::async` ile asenkron görev başlatma
- `std::future<T>` ile sonuç alma
- `launch::async` vs `launch::deferred`
- `std::promise` ile manuel değer aktarımı
- `std::shared_future` ile çoklu tüketici
- İstisna aktarımı (exception propagation)
- OBYS'de asenkron JSON export

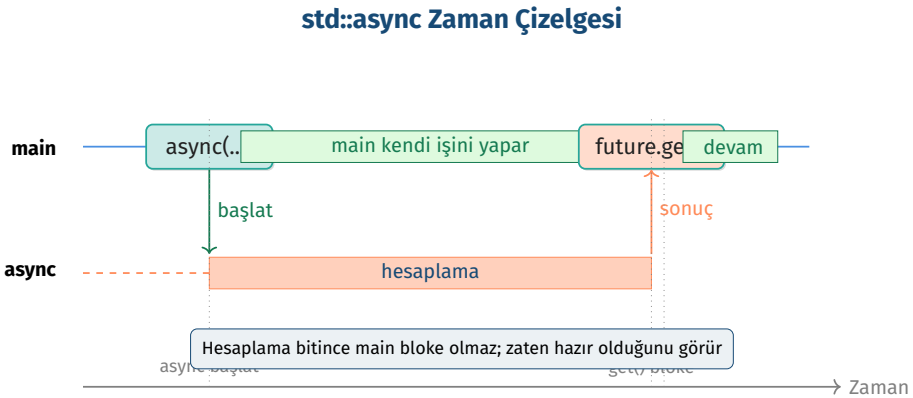
İlk async Programı

```
#include <future>

long long topla(const std::vector<int>& v) {
    return std::accumulate(v.begin(), v.end(), 0LL);
}

int main() {
    std::vector<int> sayilar(1'000'000);
    std::iota(sayilar.begin(), sayilar.end(), 1);

    auto gelecek = std::async(std::launch::async,
                              topla, std::cref(sayilar));
    std::cout << "Diger isler...\n";
    long long toplam = gelecek.get();
    std::cout << toplam << "\n";
}
```

Güzel şey

Kendi thread yönetimimizi yapmadık; `future` destructor'ı otomatik join'ler.

async Zaman Çizelgesi



future'ın gücü

Main thread bloke olmadan başka iş yapabilir; `get` çağrısı sonuç hazır değilse bekler.

std::thread'e Göre Avantajlar

1. **Otomatik yönetim**: future destructor'ı join'ler
2. **Değer döndürme**: future<T> hazır paket
3. **İstisna aktarımı**: get() sırasında yeniden fırlatılır

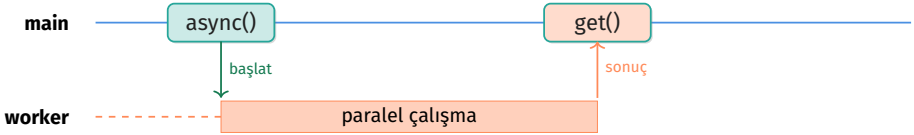
İstisna örneği

```
auto f = std::async(std::launch::async, [] {
    if (hataDurumu) throw std::runtime_error("!");
    return 42;
});
try { int x = f.get(); }
catch (const std::exception& e) { ... }
```

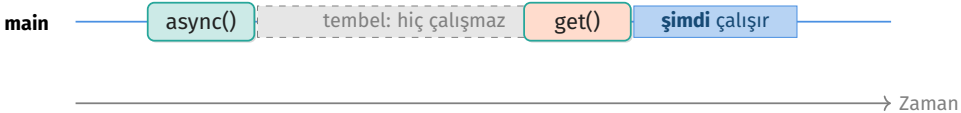
Launch Policy: async vs deferred

launch::async vs launch::deferred

std::launch::async – Yeni izlekte HEMEN çalışır



std::launch::deferred – get() çağrılana kadar HİÇ çalışmaz



launch Policy Karşılaştırması

launch::async

- Yeni izlek, hemen
- Gerçek paralellik
- `wait_for(0)` → `timeout` veya `ready`

launch::deferred

- Çağırana kadar pasif
- Paralellik yok
- `wait_for(0)` → `deferred`

Kural

Politikayı **açıkça** belirt. Varsayılan davranışa güvenme.

Sık Yapılan Hata: Geçici future

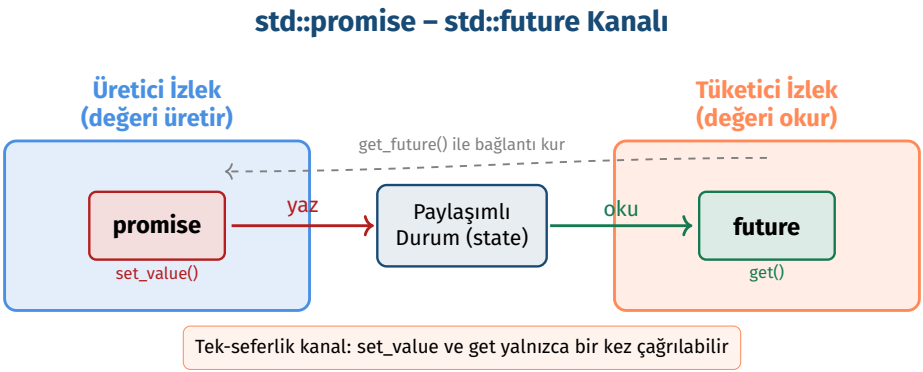
Tehlike

```
// YANLIS: Gecici future hemen yikilir,
// destructor is bitene kadar bekler -> senkron!
std::async(std::launch::async, gorev);

// DOGRU: Future'i sakla
auto f = std::async(std::launch::async, gorev);
```

Neden?

`launch::async` policy ile async'in dönen future destructor'ı, göreve `wait` ile bekler. Değişkene atamazsan anında destructor tetiklenir ve bloke olur.

std::promise – Manuel Future Kanalı



promise Kullanımı

```
void hesapla(std::promise<int> soz, int x) {
    if (x < 0) {
        soz.set_exception(std::make_exception_ptr(
            std::invalid_argument("negatif!")));
    } else {
        soz.set_value(x * x);
    }
}

int main() {
    std::promise<int> p;
    auto f = p.get_future();
    std::thread t(hesapla, std::move(p), 5);
    int sonuc = f.get();  // 25
    t.join();
}
```

Neden promise?

Kendi thread'ini yazıyorsan ama future API'sinden faydalanmak istiyorsan.

packaged_task ile Paketleme

Ne yapar?

Bir çağrılabiliri sarar; içinde otomatik promise tutar. Thread havuzu tasarımında kritik.

```
std::packaged_task<int(int, int)> task(hesapla);
std::future<int> f = task.get_future();
std::thread t(std::move(task), 10, 20);
int sonuc = f.get();  // 30
t.join();
```

shared_future – Çoklu Tüketici

```
std::promise<int> p;
std::shared_future<int> sf = p.get_future().share();

auto t = [sf](int id) {
    std::cout << id << ": " << sf.get() << "\n";
};
std::thread t1(t, 1), t2(t, 2), t3(t, 3);
p.set_value(777);  // Hepsi 777 okur
```

Kullanımlar

Dağıtılmış olay sistemleri, bir hesaplamanın sonucunu N izleğe dağıtma.

OBYS: Asenkron JSON Export

Senaryo

10.000 öğrenci JSON'a aktarılıyor. UI donmamalı, progress gösterilmeli, iptal çalışmalı.

Mimari

- `async(launch::async, exportCalistir, ...)`
- `std::atomic<size_t> islenen_` sayaç
- `std::atomic<bool> iptalBayragi_`
- UI polling: `wait_for(100ms)` + progress bar

Ünite 2 Özet

Araç seçimi

- Tek async görev → `std::async(launch::async, ...)`
- Kendi thread + future → `std::promise`
- Havuz kullanımı → `std::packaged_task`
- Çoklu tüketici → `std::shared_future`
- Binlerce görev → Ünite 3'te: thread havuzu

Ünite 3: Thread Havuzu Tasarımı

Ünite 3 – Öğrenme Hedefleri

- Thread havuzu neden gerekir?
- `hardware_concurrency` ile işçi sayısı
- Thread-safe iş kuyruğu (`ThreadSafeQueue<T>`)
- `packaged_task` ile tip-silme
- Sıfırdan 50 satır `ThreadPool` sınıfı
- OBYS'de 10.000 öğrenci paralel GPA

Neden Thread Havuzu?

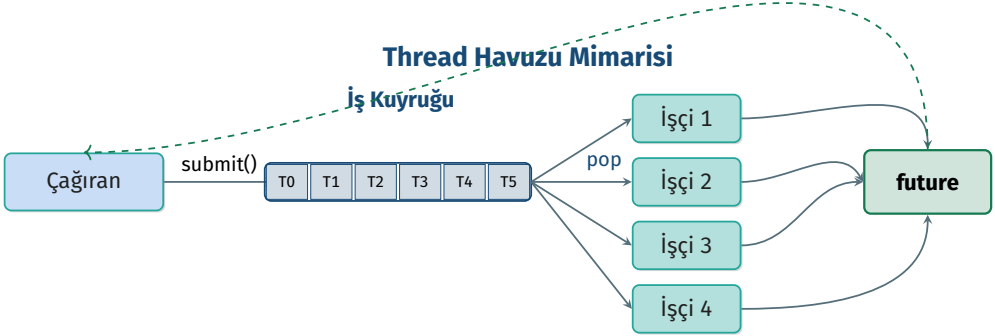
std::thread başına maliyet

- Oluşturma: 10-100 mikrosaniye
- Yığın (stack): 1-8 MB RAM
- Bağlam değişimi: her geçişte cache miss

10.000 görev $\times$ 3 ms

- Her görev için ayrı thread: **500 ms overhead**
- Thread havuzu (8 işçi): **410 ms toplam**

Havuz Mimarisi



hardware_concurrency

```
unsigned n = std::thread::hardware_concurrency();
// 4, 8, 16, 32 -- mantiksal cekirdek
if (n == 0) n = 4;  // fallback
```

Kaç işçi?

- **CPU-yoğun**: çekirdek sayısı
- **I/O-yoğun**: 2-4x çekirdek
- **Karma**: biraz daha fazla

En iyi değer için benchmark yap.

ThreadSafeQueue Sınıfı

```
template <typename T>
class ThreadSafeQueue {
public:
    void push(T v) {
        { std::lock_guard lk(mtx_);
          if (kap_) return;
          q_.push(std::move(v)); }
        cv_.notify_one();
    }
    bool wait_and_pop(T& out) {
        std::unique_lock lk(mtx_);
        cv_.wait(lk, [this] {
            return !q_.empty() || kap_; });
        if (q_.empty()) return false;
        out = std::move(q_.front()); q_.pop();
        return true;
    }
    void close() { kap_ = true; cv_.notify_all(); }
private:
    std::mutex mtx_;
    std::queue<T> q_;
    std::condition_variable cv_;
    bool kap_ = false;
};
```

ThreadPool Sınıfı – Ana Fikir

```
class ThreadPool {
public:
    explicit ThreadPool(size_t n) {
        for (size_t i = 0; i < n; ++i)
            isciler_.emplace_back([this] {
                std::function<void()> g;
                while (kuyruk_.wait_and_pop(g)) g();
            });
    }
    ~ThreadPool() {
        kuyruk_.close();
        for (auto& t : isciler_) if (t.joinable()) t.join();
    }
    template<typename F, typename... A>
    auto submit(F&& f, A&&... a)
        -> std::future<std::invoke_result_t<F, A...>>;
private:
    std::vector<std::thread> isciler_;
    ThreadSafeQueue<std::function<void()>> kuyruk_;
};
```

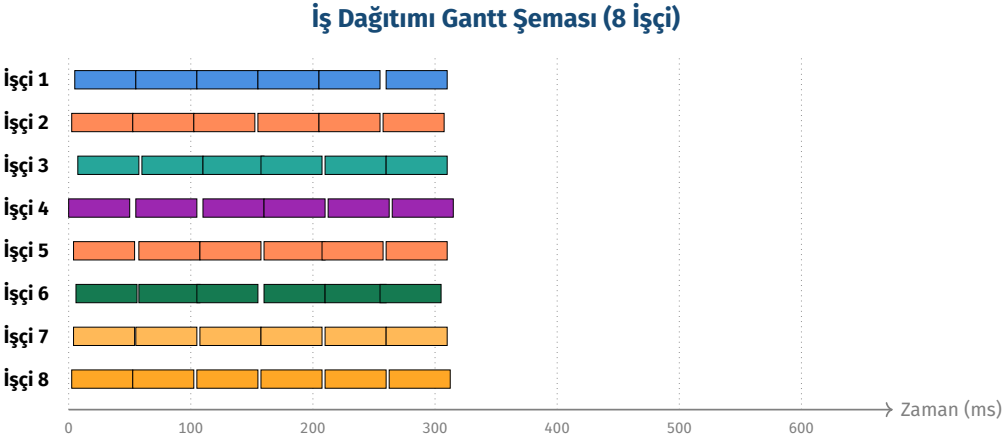
submit – Şablon Büyüsü

```
template<typename F, typename... A>
auto ThreadPool::submit(F&& f, A&&... a)
    -> std::future<std::invoke_result_t<F, A...>>
{
    using R = std::invoke_result_t<F, A...>;
    auto gorev = std::make_shared<std::packaged_task<R()>>(
        std::bind(std::forward<F>(f), std::forward<A>(a)...));
    std::future<R> gelecek = gorev->get_future();
    kuyruk_.push([gorev] { (*gorev)(); });
    return gelecek;
}
```

Niçin shared_ptr?

`packaged_task` taşınabilir ama kopyalanamaz; `std::function` kopyalanabilir ister. `shared_ptr` bu boşluğu doldurur.

İş Dağıtımı



Her bar bir görev; 100 görev 8 işçi arasında paralel dağıtılır, ∼625 ms'de tamamlanır

OBYS: Toplu Not Hesaplama

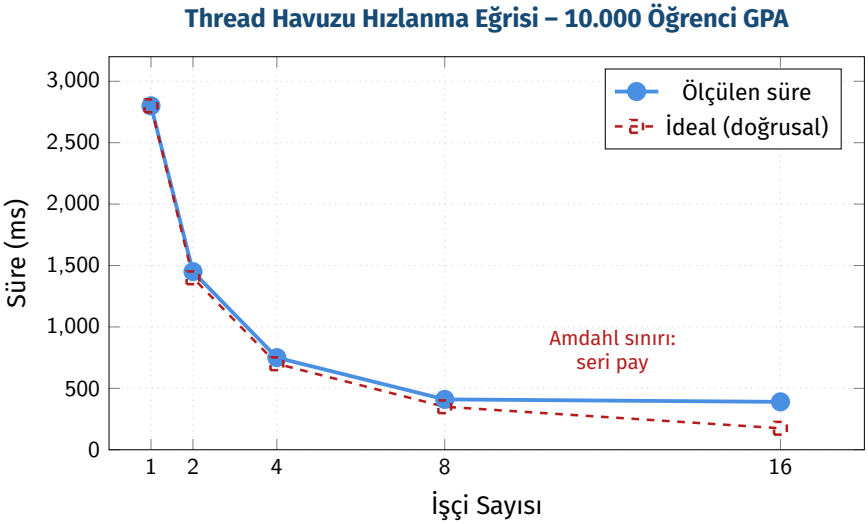
Senaryo

10.000 öğrenci dönem sonu GPA hesaplama. Her öğrenci 0.3-3 ms sürüyor.

Kullanım

```
ThreadPool havuz(8);
std::vector<std::future<GPASonuc>> f;
for (const auto& o : ogrenciler)
    f.push_back(havuz.submit(hesaplaGPA, std::cref(o)));
std::vector<GPASonuc> sonuc;
for (auto& fu : f) sonuc.push_back(fu.get());
```

Performans Ölçüm



Granülerlik ve Batch Optimizasyonu

Küçük görevler sorun yaratır

10.000 görev $\times$ 0.3 ms = toplam 3 saniye iş.
10.000 submit $\times$ 10 μs = 100 ms overhead.

Çözüm: batch (paket)

- 100'lük paketler $\rightarrow$ 100 görev
- Overhead: 100 $\times$ 10 μs = 1 ms
- Kazanç: %30-50

Deadlock Tehlikesi

İç içe submit + get

```
auto f1 = havuz.submit([&] {
    auto f2 = havuz.submit([] { return is(); });
    return f2.get();  // Tehlike!
});
```

Neden?

Yeterli işçi kalmazsa, f2 çalıştıracak işçi bulamaz. Tüm işçiler f1'lerde get beklerse **deadlock**.

Ünite 3 Özet

Yaygın hatalar

- Her iş için yeni havuz yaratmak
- İşçi sayısının çekirdekten çok fazla olması
- İç içe submit + get
- Destructor'da join'i unutmak

Başarı

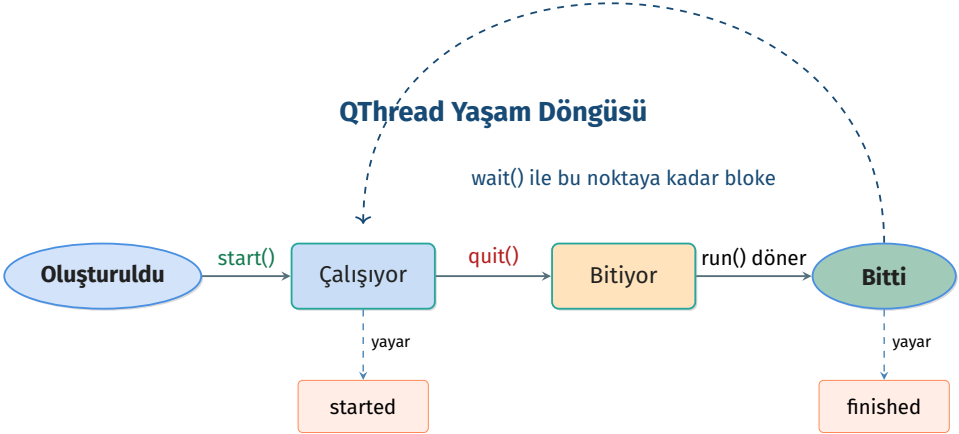
OBYS'de 2.8s $\rightarrow$ 0.4s (7x hızlanma).

Ünite 4: Qt ile Çok Kanallı Programlama

Ünite 4 – Öğrenme Hedefleri

- `QThread` yaşam döngüsü
- Alt-sınıflama yanılgısı
- Worker Pattern (`moveToThread`)
- `Qt::QueuedConnection` ve thread sınırı
- `QtConcurrent::run` ve `::mapped`
- `QFuture`, `QFutureWatcher`, `QProgressDialog`
- OBYS responsive UI

QThread Yaşam Döngüsü



QThread Alt-Sınıflama Yanılgısı

Eski örnek (KAÇIN)

```
class Hesap : public QThread {
    void run() override { ... }
};
```

Sorun

QThread nesnesi ana izlekte yaşar. Alt sınıfa eklenen slot'lar ana izlekte çalışır, run() ise arka planda. Karışık ve hata-üreten.

Çözüm: Worker Pattern

QObject worker + moveToThread + sinyal/slot.

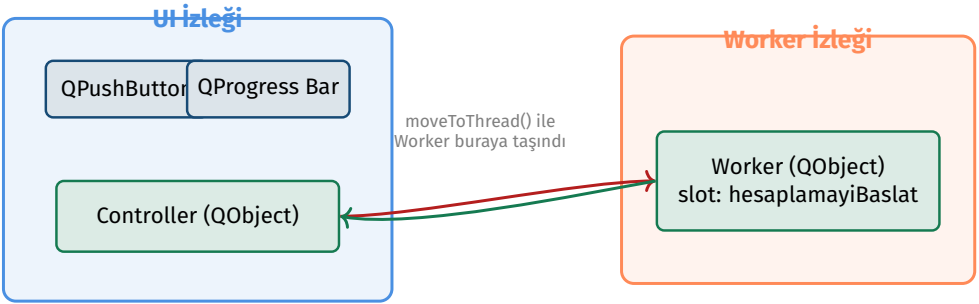
Qt Connection Türleri

- `DirectConnection`: slot sinyal izleğinde hemen
- `QueuedConnection`: slot alıcı izleğinde, kuyruğa konur
- `BlockingQueuedConnection`: queued + yayıcı bekler
- `AutoConnection` (varsayılan): aynı izlekte Direct, farklıda Queued
- `UniqueConnection`: duplicate önlemi

Thread-safe haberleşme

`QueuedConnection` Qt'nin thread sınırı geçişinin *sihri*. Tüm marshalling otomatik.

Worker Pattern: moveToThread



Qt::QueuedConnection – sinyaller olay kuyruğuna konur, alıcı izlekte çalıştırılır

emit komut

emit sonuç/ilerleme

Worker Sınıfı

```
class HesaplamaWorker : public QObject {
    Q_OBJECT
public slots:
    void hesaplamayiBaslat(QVector<Ogrenci> v) {
        for (int i = 0; i < v.size(); ++i) {
            QCoreApplication::processEvents();
            if (iptal_.load()) { emit iptalEdildi(); return; }
            double g = hesaplaGPA(v[i]);
            emit ilerlemeDegisti(i + 1, v.size());
            emit sonucHazir(v[i].numara, g);
        }
        emit tumSonuclarHazir();
    }
signals:
    void ilerlemeDegisti(int, int);
    void sonucHazir(int, double);
    void tumSonuclarHazir();
    void iptalEdildi();
};
```

Controller Kurulumu

```
izlek_ = new QThread(this);
worker_ = new HesaplamaWorker();  // parent YOK!
worker_->moveToThread(izlek_);

connect(this, &Controller::baslat,
        worker_, &Worker::hesaplamayiBaslat);
connect(worker_, &Worker::sonucHazir,
        this, &Controller::sonucHazir);
connect(izlek_, &QThread::finished,
        worker_, &QObject::deleteLater);

izlek_->start();
```

Kritik

Worker'a parent verme; moveToThread başka izleğe taşıma yapacak. AutoConnection → farklı izlekler için Queued olur.

QtConcurrent – Hafif Alternatif

QtConcurrent::run

```
QFuture<int> f = QtConcurrent::run(agirHesaplama, 10);
int sonuc = f.result();  // veya QFutureWatcher
```

QtConcurrent::mapped – Paralel map

```
QFuture<GPASonuc> f =
    QtConcurrent::mapped(ogrenciler, hesaplaGPA);
```

Avantaj

Qt global thread havuzunu kullanır; otomatik paralelleştirme.

QFuture vs std::future

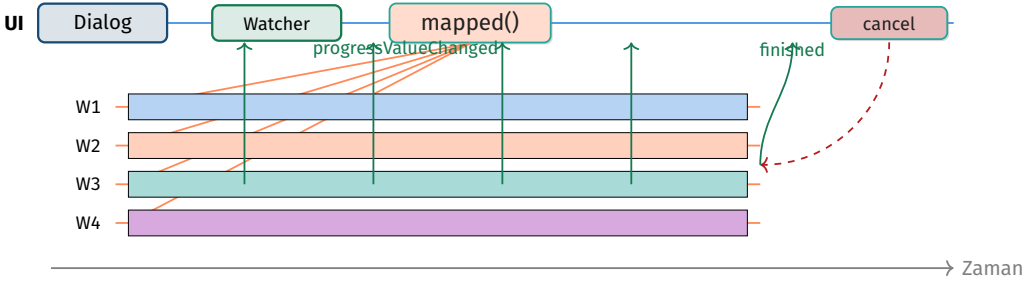
Özellik	QFuture	std::future
Tek seferlik	Hayır	Evet
İlerleme takibi	Evet	Hayır
İptal	Evet	Yok
Signal/slot	Watcher ile	Yok
Portable	Qt bağımlı	Evet

Qt uygulamasında

QFuture + QFutureWatcher UI entegrasyonu için mükemmeldir.

QProgressDialog Akışı

QProgressDialog + QFutureWatcher Akışı



Watcher'ın sinyalleri doğrudan Dialog slot'larına bağlanır; UI canlı kalır

iptal

OBYS Responsive UI – Tam Kod

```
void RaporPenceresi::raporuBaslat() {
    progressDialog_ = new QProgressDialog(
        "Hesaplaniyor...", "Iptal", 0, ogrenciler_.size(), this);
    progressDialog_->setWindowModality(Qt::WindowModal);

    watcher_ = new QFutureWatcher<GPASonuc>(this);
    connect(watcher_, &QFutureWatcher<GPASonuc>::progressValueChanged,
            progressDialog_, &QProgressDialog::setValue);
    connect(watcher_, &QFutureWatcher<GPASonuc>::finished,
            this, &RaporPenceresi::raporBitti);
    connect(progressDialog_, &QProgressDialog::canceled,
            watcher_, &QFutureWatcher<GPASonuc>::cancel);

    QFuture<GPASonuc> gelecek = QtConcurrent::mapped(
        ogrenciler_, hesaplaGPA);
    watcher_->setFuture(gelecek);
}
```

Qt Thread Araç Seçim Rehberi

Senaryo	Araç
Tek, kısa asenkron iş	`QtConcurrent::run`
Vektör üzerinde paralel işlem	`QtConcurrent::mapped`
Karmaşık state + sinyal/slot	Worker Pattern
Düşük seviye kontrol	QThread + QRunnable
Saf C++ (Qt olmayan)	`std::thread/async`

Ünite 4 Özet

Yaygın hatalar

- QThread alt-sınıflama (anti-pattern)
- moveToThread öncesi parent verme
- UI widget'ını arka plan izleğinden güncelleme
- Destructor'da `quit` + `wait` atmamak
- `terminate` kullanma

Başarı

OBYS'de 10.000 öğrenci GPA'sı arka planda hesaplanıyor, progress dialog canlı, iptal çalışıyor, UI donmuyor.

Bu Haftanın Kazanımları

Öğrendiklerimiz

- `std::condition_variable` ile izlek koordinasyonu
- `std::async`, `future`, `promise` üst seviye API
- Sıfırdan bir modern thread havuzu
- Qt'nin worker ve QtConcurrent paradigmaları

OBYS İlerlemesi

Toplu yükleme, asenkron export, paralel GPA, responsive rapor – hepsi çalışıyor ve UI hiç donmuyor.

Sonraki Hafta

Bölüm 11: Eşzamanlılık Desenleri ve OBYS

Şimdiye kadar öğrendiklerimizi endüstri standardı desenlere oturtacağız:

- Producer-Consumer
- Active Object
- Monitor
- Scatter-Gather

OBYS'nin tüm çok kanallı mimarisini tek bir tutarlı bütün haline getireceğiz.

Sonra Bölüm 12: Ağ

OBYS bir uygulama olmaktan çıkıp sistem olacak: istemci-sunucu mimarisi, TCP, Qt Network.

Teşekkürler

Bu haftalık bu kadar!

Sorular?

Soruyu takip etmek için:
`BILB2000 - Yazılım Geliştirme II`
Yrd. Doç. Dr. Tolga BERBER